

fintxter Book: Simplicity - The Finer Art of Creating Software

Complexity

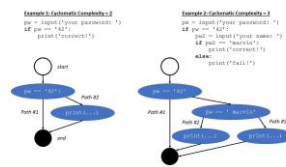
"A whole, made up of parts—difficult to analyze, understand, or explain".

- Complexity appears in
- Project Lifecycle
 - Code Development
 - Algorithmic Theory
 - Processes
 - Social Networks
 - Learning & Your Daily Life

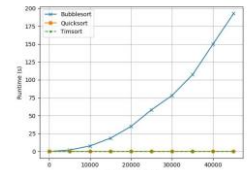
Project Lifecycle



Cyclomatic Complexity



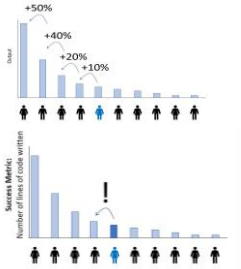
Runtime Complexity



→ Complexity reduces productivity and focus. It'll consume your precious time. **Keep it simple!**

80/20 Principle

Majority of effects come from the minority of causes.



Pareto Tips

1. Figure out your success metrics.
2. Figure out your big goals in life.
3. Look for ways to achieve the same things with fewer resources.
4. Reflect on your own successes
5. Reflect on your own failures
6. Read more books in your industry.
7. Spend much of your time improving and tweaking existing products
8. Smile.
9. Don't do things that reduce value

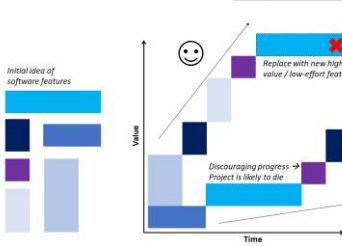
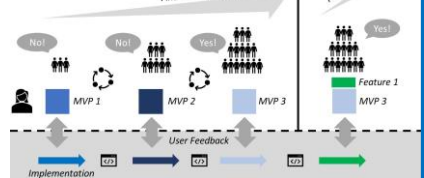
Maximize Success Metric:

#lines of code written

Minimum Viable Product (MVP)

A minimum viable product in the software sense is code that is stripped from all features to focus on the core functionality.

Minimum Viable Product & Iterative Feedback Loop



How to MVP?

- Formulate hypothesis
- Omit needless features
- Split test to validate each new feature
- Focus on product-market fit
- Seek high-value and low-cost features

Clean Code Principles

1. You Ain't Going to Need It
2. The Principle of Least Surprise
3. Don't Repeat Yourself
4. **Code For People Not Machines**
5. Stand on the Shoulders of Giants
6. Use the Right Names
7. Single-Responsibility Principle
8. Use Comments
9. Avoid Unnecessary Comments
10. Be Consistent
11. Test
12. Think in Big Pictures
13. Only Talk to Your Friends
14. Refactor
15. Don't Overengineer
16. Don't Overuse Indentation
17. Small is Beautiful
18. Use Metrics
19. Boy Scout Rule: Leave Camp Cleaner Than You Found It

Unix Philosophy

1. Simple's Better Than Complex
2. **Small is Beautiful (Again)**
3. Make Each Program Do One Thing Well
4. Build a Prototype First
5. Portability Over Efficiency
6. Store Data in Flat Text Files
7. Use Software Leverage
8. Avoid Captive User Interfaces
9. **Program = Filter**
10. Worse is Better
11. Clean > Clever Code
12. **Design Connected Programs**
13. Make Your Code Robust
14. Repair What You Can — But Fail Early and Noisily
15. Write Programs to Write Programs

Premature Optimization

"Programmers waste enormous amounts of time thinking about [...] the speed of noncritical parts of their programs. We should forget about small efficiencies, say about 97% of the time: **premature optimization is the root of all evil.**" — Donald Knuth

Performance Tuning 101

1. Measure, then improve
2. Focus on the slow 20%
3. Algorithmic optimization wins
4. All hail to the cache
5. Solve an easier problem version
6. Know when to stop

Less Is More in Design

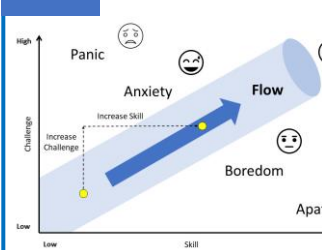


How to Simplify Design?

1. Use whitespace
2. Remove design elements
3. Remove features
4. Reduce variation of fonts, font types, colors
5. Be consistent across UIs

Flow

"... the source code of ultimate human performance" — Kotler



How to Achieve Flow? (1) clear goals, (2) immediate feedback, and (3) balance opportunity & capacity.

Flow Tips for Coders

1. Always work on an explicit practical code project
2. Work on fun projects that fulfill your purpose
3. Perform from your strengths
4. Big chunks of coding time
5. Reduce distractions: smartphone + social
6. Sleep a lot, eat healthily, read quality books, and exercise → garbage in, garbage out!

Focus

You can take raw resources and move them from a state of high entropy into a state of low entropy—using **focused effort towards the attainment of a greater plan.**



3-Step Approach of Efficient Software Creation

1. Plan your code
2. Apply focused effort to make it real.
3. Seek feedback

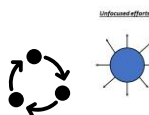


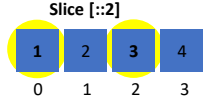
Figure: Same effort, different result.

Subscribe to the **11x FREE** Python Cheat Sheet Course:
<https://blog.fintxter.com/python-cheat-sheets/>

Keywords

Keyword	Description	Code Examples
<code>False</code> , <code>True</code>	Boolean data type	<code>False == (1 > 2)</code> <code>True == (2 > 1)</code> 
<code>and</code> , <code>or</code> , <code>not</code>	Logical operators → Both are true → Either is true → Flips Boolean	<code>True and True</code> # True <code>True or False</code> # True <code>not False</code> # True
<code>break</code>	Ends loop prematurely	<code>while True:</code> <code>break</code> # finite loop
<code>continue</code>	Finishes current loop iteration	<code>while True:</code> <code>continue</code> <code>print("42")</code> # dead code
<code>class</code>	Defines new class	<code>class Coffee:</code> # Define your class
<code>def</code>	Defines a new function or class method.	<code>def say_hi():</code> <code>print('hi')</code>
<code>if</code> , <code>elif</code> , <code>else</code>	Conditional execution: - "if" condition == True? - "elif" condition == True? - Fallback: else branch	<code>x = int(input("ur val:"))</code> <code>if x > 3: print("Big")</code> <code>elif x == 3: print("3")</code> <code>else: print("Small")</code>
<code>for</code> , <code>while</code>	# For loop <code>for i in [0,1,2]:</code> <code>print(i)</code>	# While loop does same <code>j = 0</code> <code>while j < 3:</code> <code>print(j); j = j + 1</code> 
<code>in</code>	Sequence membership	<code>42 in [2, 39, 42]</code> # True
<code>is</code>	Same object memory location	<code>y = x = 3</code> <code>x is y</code> # True <code>[3] is [3]</code> # False
<code>None</code>	Empty value constant	<code>print()</code> is None # True
<code>lambda</code>	Anonymous function	<code>(lambda x: x+3)(3)</code> # 6 
<code>return</code>	Terminates function. Optional return value defines function result.	<code>def increment(x):</code> <code>return x + 1</code> <code>increment(4)</code> # returns 5

Basic Data Structures

Type	Description	Code Examples
Boolean	The Boolean data type is either <code>True</code> or <code>False</code> . Boolean operators are ordered by priority: <code>not</code> → <code>and</code> → <code>or</code> <code>{}</code> → <code>{1, 2, 3}</code> 	<code>## Evaluates to True:</code> <code>1<2 and 0<=1 and 3>2 and 2>=2 and 1==1 and 1!=0</code> <code>## Evaluates to False:</code> <code>bool(None or 0 or 0.0 or '' or [] or {} or set())</code> Rule: <code>None</code> , <code>0</code> , <code>0.0</code> , empty strings, or empty container types evaluate to <code>False</code>
Integer, Float	An integer is a positive or negative number without decimal point such as 3. A float is a positive or negative number with floating point precision such as 3.1415926. Integer division rounds toward the smaller integer (example: <code>3//2==1</code>).	<code>## Arithmetic Operations</code> <code>x, y = 3, 2</code> <code>print(x + y)</code> # 5 <code>print(x - y)</code> # 1 <code>print(x * y)</code> # 6 <code>print(x / y)</code> # 1.5 <code>print(x // y)</code> # 1 <code>print(x % y)</code> # 1 <code>print(-x)</code> # -3 <code>print(abs(-x))</code> # 3 <code>print(int(3.9))</code> # 3 <code>print(float(3))</code> # 3.0 <code>print(x ** y)</code> # 9 
String	Python Strings are sequences of characters. String Creation Methods: 1. Single quotes >>> 'Yes' 2. Double quotes >>> "Yes" 3. Triple quotes (multi-line) >>> """Yes We Can""" 4. String method >>> str(5) == '5' True 5. Concatenation >>> "Ma" + "hatma" 'Mahatma' Whitespace chars: Newline \n, Space \s, Tab \t	<code>## Indexing and Slicing</code> <code>s = "The youngest pope was 11 years"</code> <code>s[0]</code> # 'T' <code>s[1:3]</code> # 'he' <code>s[-3:-1]</code> # 'ar' <code>s[-3:]</code> # 'ars'  <code>x = s.split()</code> <code>x[-2] + " " + x[2] + "s" # '11 popes'</code> <code>## String Methods</code> <code>y = " Hello world\t\n"</code> <code>y.strip()</code> # Remove Whitespace <code>"HI".lower()</code> # Lowercase: 'hi' <code>"hi".upper()</code> # Uppercase: 'HI' <code>"hello".startswith("he")</code> # True <code>"hello".endswith("lo")</code> # True <code>"hello".find("ll")</code> # Match at 2 <code>"cheat".replace("ch", "m")</code> # 'meat' <code>''.join(["F", "B", "I"])</code> # 'FBI' <code>len("hello world")</code> # Length: 15 <code>"ear" in "earth"</code> # True

Complex Data Structures

Type	Description	Example
List	Stores a sequence of elements. Unlike strings, you can modify list objects (they're <i>mutable</i>).	<code>l = [1, 2, 2]</code> <code>print(len(l))</code> # 3 
Adding elements	Add elements to a list with (i) <code>append</code> , (ii) <code>insert</code> , or (iii) list concatenation.	<code>[1, 2].append(4)</code> # [1, 2, 4] <code>[1, 4].insert(1,9)</code> # [1, 9, 4] <code>[1, 2] + [4]</code> # [1, 2, 4]
Removal	Slow for lists	<code>[1, 2, 2, 4].remove(1)</code> # [2, 2, 4]
Reversing	Reverses list order	<code>[1, 2, 3].reverse()</code> # [3, 2, 1]
Sorting	Sorts list using fast Timsort	<code>[2, 4, 2].sort()</code> # [2, 2, 4]
Indexing	Finds the first occurrence of an element & returns index. Slow worst case for whole list traversal.	<code>[2, 2, 4].index(2)</code> # index of item 2 is 0 <code>[2, 2, 4].index(2,1)</code> # index of item 2 after pos 1 is 1
Stack	Use Python lists via the list operations <code>append()</code> and <code>pop()</code>	<code>stack = [3]</code> <code>stack.append(42)</code> # [3, 42] <code>stack.pop()</code> # 42 (stack: [3]) <code>stack.pop()</code> # 3 (stack: [])
Set	An unordered collection of unique elements (<i>at-most-once</i>) → fast membership <i>O(1)</i>	<code>basket = {'apple', 'eggs', 'banana', 'orange'}</code> <code>same = set(['apple', 'eggs', 'banana', 'orange'])</code>

Type	Description	Example
Dictionary	Useful data structure for storing (key, value) pairs	<code>cal = {'apple': 52, 'banana': 89, 'choco': 546}</code> # calories
Reading and writing elements	Read and write elements by specifying the key within the brackets. Use the <code>keys()</code> and <code>values()</code> functions to access all keys and values of the dictionary	<code>print(cal['apple'] < cal['choco'])</code> # True <code>cal['cappu'] = 74</code> <code>print(cal['banana'] < cal['cappu'])</code> # False <code>print('apple' in cal.keys())</code> # True <code>print(52 in cal.values())</code> # True
Dictionary Iteration	You can access the (key, value) pairs of a dictionary with the <code>items()</code> method.	<code>for k, v in cal.items():</code> <code>print(k) if v > 500 else ''</code> # 'choco'
Membership operator	Check with the <code>in</code> keyword if set, list, or dictionary contains an element. Set membership is faster than list membership.	<code>basket = {'apple', 'eggs', 'banana', 'orange'}</code> <code>print('eggs' in basket)</code> # True <code>print('mushroom' in basket)</code> # False
List & set comprehension	List comprehension is the concise Python way to create lists. Use brackets plus an expression, followed by a <code>for</code> clause. Close with zero or more <code>for</code> or <code>if</code> clauses. Set comprehension works similar to list comprehension.	<code>l = ['hi' + x for x in ['Alice', 'Bob', 'Pete']]</code> # ['Hi Alice', 'Hi Bob', 'Hi Pete'] <code>l2 = [x * y for x in range(3) for y in range(3) if x>y]</code> # [0, 0, 2] <code>squares = {x**2 for x in [0,2,4] if x < 4}</code> # {0, 4}



Python Cheat Sheet: Keywords

“A puzzle a day to learn, code, and play” → Visit finxter.com

Keyword	Description	Code example
<code>False, True</code>	Data values from the data type Boolean	<code>False == (1 > 2), True == (2 > 1)</code>
<code>and, or, not</code>	Logical operators: (<code>x and y</code>) → both x and y must be True (<code>x or y</code>) → either x or y must be True (<code>not x</code>) → x must be false	<pre>x, y = True, False (x or y) == True # True (x and y) == False # True (not y) == True # True</pre>
<code>break</code>	Ends loop prematurely	<pre>while(True): break # no infinite loop print("hello world")</pre>
<code>continue</code>	Finishes current loop iteration	<pre>while(True): continue print("43") # dead code</pre>
<code>class</code> <code>def</code>	Defines a new class → a real-world concept (object oriented programming) Defines a new function or class method. For latter, first parameter (“self”) points to the class object. When calling class method, first parameter is implicit.	<pre>class Beer: def __init__(self): self.content = 1.0 def drink(self): self.content = 0.0 becks = Beer() # constructor - create class becks.drink() # beer empty: b.content == 0</pre>
<code>if, elif, else</code>	Conditional program execution: program starts with “if” branch, tries the “elif” branches, and finishes with “else” branch (until one branch evaluates to True).	<pre>x = int(input("your value: ")) if x > 3: print("Big") elif x == 3: print("Medium") else: print("Small")</pre>
<code>for, while</code>	<pre># For loop declaration for i in [0,1,2]: print(i)</pre>	<pre># While loop - same semantics j = 0 while j < 3: print(j) j = j + 1</pre>
<code>in</code>	Checks whether element is in sequence	<code>42 in [2, 39, 42] # True</code>
<code>is</code>	Checks whether both elements point to the same object	<pre>y = x = 3 x is y # True [3] is [3] # False</pre>
<code>None</code>	Empty value constant	<pre>def f(): x = 2 f() is None # True</pre>
<code>lambda</code>	Function with no name (anonymous function)	<code>(lambda x: x + 3)(3) # returns 6</code>
<code>return</code>	Terminates execution of the function and passes the flow of execution to the caller. An optional value after the return keyword specifies the function result.	<pre>def incrementor(x): return x + 1 incrementor(4) # returns 5</pre>

Python Cheat Sheet: Basic Data Types

“A puzzle a day to learn, code, and play” → Visit finxter.com

	Description	Example
Boolean	The Boolean data type is a truth value, either True or False. The Boolean operators ordered by priority: not x → “if x is False, then x, else y” x and y → “if x is False, then x, else y” x or y → “if x is False, then y, else x” These comparison operators evaluate to True: 1 < 2 and 0 <= 1 and 3 > 2 and 2 >= 2 and 1 == 1 and 1 != 0 # True	<pre>## 1. Boolean Operations x, y = True, False print(x and not y) # True print(not x and y or x) # True ## 2. If condition evaluates to False if None or 0 or 0.0 or '' or [] or {} or set(): # None, 0, 0.0, empty strings, or empty # container types are evaluated to False print("Dead code") # Not reached</pre>
Integer, Float	An integer is a positive or negative number without floating point (e.g. 3). A float is a positive or negative number with floating point precision (e.g. 3.14159265359). The // operator performs integer division. The result is an integer value that is rounded toward the smaller integer number (e.g. 3 // 2 == 1).	<pre>## 3. Arithmetic Operations x, y = 3, 2 print(x + y) # = 5 print(x - y) # = 1 print(x * y) # = 6 print(x / y) # = 1.5 print(x // y) # = 1 print(x % y) # = 1s print(-x) # = -3 print(abs(-x)) # = 3 print(int(3.9)) # = 3 print(float(3)) # = 3.0 print(x ** y) # = 9</pre>
String	Python Strings are sequences of characters. The four main ways to create strings are the following. 1. Single quotes 'Yes' 2. Double quotes "Yes" 3. Triple quotes (multi-line) """Yes We Can""" 4. String method str(5) == '5' # True 5. Concatenation "Ma" + "hatma" # 'Mahatma' These are whitespace characters in strings. • Newline \n • Space \s • Tab \t	<pre>## 4. Indexing and Slicing s = "The youngest pope was 11 years old" print(s[0]) # 'T' print(s[1:3]) # 'he' print(s[-3:-1]) # 'ol' print(s[-3:]) # 'old' x = s.split() # creates string array of words print(x[-3] + " " + x[-1] + " " + x[2] + "s") # '11 old popes' ## 5. Most Important String Methods y = " This is lazy\t\n " print(y.strip()) # Remove Whitespace: 'This is lazy' print("DrDre".lower()) # Lowercase: 'drdre' print("attention".upper()) # Uppercase: 'ATTENTION' print("smartphone".startswith("smart")) # True print("smartphone".endswith("phone")) # True print("another".find("other")) # Match index: 2 print("cheat".replace("ch", "m")) # 'meat' print(','.join(["F", "B", "I"])) # 'F,B,I' print(len("Rumpelstiltskin")) # String length: 15 print("ear" in "earth") # Contains: True</pre>

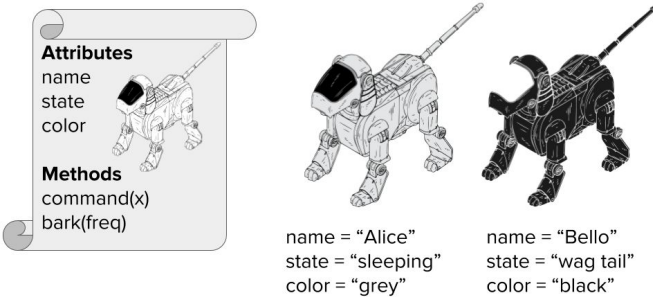
Python Cheat Sheet: Complex Data Types

“A puzzle a day to learn, code, and play” → Visit finxter.com

	Description	Example
List	A container data type that stores a sequence of elements. Unlike strings, lists are mutable: modification possible.	<pre>l = [1, 2, 2] print(len(l)) # 3</pre>
Adding elements	Add elements to a list with (i) append, (ii) insert, or (iii) list concatenation. The append operation is very fast.	<pre>[1, 2, 2].append(4) # [1, 2, 2, 4] [1, 2, 4].insert(2,2) # [1, 2, 2, 4] [1, 2, 2] + [4] # [1, 2, 2, 4]</pre>
Removal	Removing an element can be slower.	<pre>[1, 2, 2, 4].remove(1) # [2, 2, 4]</pre>
Reversing	This reverses the order of list elements.	<pre>[1, 2, 3].reverse() # [3, 2, 1]</pre>
Sorting	Sorts a list. The computational complexity of sorting is linear in the no. list elements.	<pre>[2, 4, 2].sort() # [2, 2, 4]</pre>
Indexing	Finds the first occurrence of an element in the list & returns its index. Can be slow as the whole list is traversed.	<pre>[2, 2, 4].index(2) # index of element 4 is "0" [2, 2, 4].index(2,1) # index of element 2 after pos 1 is "1"</pre>
Stack	Python lists can be used intuitively as stacks via the two list operations append() and pop().	<pre>stack = [3] stack.append(42) # [3, 42] stack.pop() # 42 (stack: [3]) stack.pop() # 3 (stack: [])</pre>
Set	A set is an unordered collection of unique elements (“at-most-once”).	<pre>basket = {'apple', 'eggs', 'banana', 'orange'} same = set(['apple', 'eggs', 'banana', 'orange'])</pre>
Dictionary	The dictionary is a useful data structure for storing (key, value) pairs.	<pre>calories = {'apple' : 52, 'banana' : 89, 'choco' : 546}</pre>
Reading and writing elements	Read and write elements by specifying the key within the brackets. Use the keys() and values() functions to access all keys and values of the dictionary.	<pre>print(calories['apple'] < calories['choco']) # True calories['cappu'] = 74 print(calories['banana'] < calories['cappu']) # False print('apple' in calories.keys()) # True print(52 in calories.values()) # True</pre>
Dictionary Looping	You can access the (key, value) pairs of a dictionary with the items() method.	<pre>for k, v in calories.items(): print(k) if v > 500 else None # 'chocolate'</pre>
Membership operator	Check with the ‘in’ keyword whether the set, list, or dictionary contains an element. Set containment is faster than list containment.	<pre>basket = {'apple', 'eggs', 'banana', 'orange'} print('eggs' in basket) # True print('mushroom' in basket) # False</pre>
List and Set Comprehension	List comprehension is the concise Python way to create lists. Use brackets plus an expression, followed by a for clause. Close with zero or more for or if clauses. Set comprehension is similar to list comprehension.	<pre># List comprehension l = [('Hi ' + x) for x in ['Alice', 'Bob', 'Pete']] print(l) # ['Hi Alice', 'Hi Bob', 'Hi Pete'] l2 = [x * y for x in range(3) for y in range(3) if x>y] print(l2) # [0, 0, 2] # Set comprehension squares = { x**2 for x in [0,2,4] if x < 4 } # {0, 4}</pre>

Python Cheat Sheet: Classes

“A puzzle a day to learn, code, and play” → Visit finxter.com

	Description	Example
Classes	A class encapsulates data and functionality: data as attributes, and functionality as methods. It is a blueprint for creating concrete instances in memory. Class Instances 	<pre>class Dog: """ Blueprint of a dog """ # class variable shared by all instances species = ["canis lupus"] def __init__(self, name, color): self.name = name self.state = "sleeping" self.color = color def command(self, x): if x == self.name: self.bark(2) elif x == "sit": self.state = "sit" else: self.state = "wag tail" def bark(self, freq): for i in range(freq): print("[" + self.name + "]: Woof!") bello = Dog("bello", "black") alice = Dog("alice", "white") print(bello.color) # black print(alice.color) # white bello.bark(1) # [bello]: Woof! alice.command("sit") print("[alice]: " + alice.state) # [alice]: sit bello.command("no") print("[bello]: " + bello.state) # [bello]: wag tail alice.command("alice") # [alice]: Woof! # [alice]: Woof! bello.species += ["wulf"] print(len(bello.species) == len(alice.species)) # True (!)</pre>
Instance	You are an instance of the class human. An instance is a concrete implementation of a class: all attributes of an instance have a fixed value. Your hair is blond, brown, or black--but never unspecified. Each instance has its own attributes independent of other instances. Yet, class variables are different. These are data values associated with the class, not the instances. Hence, all instance share the same class variable species in the example.	
Self	The first argument when defining any method is always the self argument. This argument specifies the instance on which you call the method. self gives the Python interpreter the information about the concrete instance. To <i>define</i> a method, you use self to modify the instance attributes. But to <i>call</i> an instance method, you do not need to specify self.	
Creation	You can create classes “on the fly” and use them as logical units to store complex data types. <pre>class Employee(): pass employee = Employee() employee.salary = 122000 employee.firstname = "alice" employee.lastname = "wonderland" print(employee.firstname + " " + employee.lastname + " " + str(employee.salary) + "\$") # alice wonderland 122000\$</pre>	

Python Cheat Sheet: Functions and Tricks

“A puzzle a day to learn, code, and play” → Visit finxter.com

		Description	Example	Result
ADVANCED FUNCTIONALS	map(func, iter)	Executes the function on all elements of the iterable	list(map(lambda x: x[0], ['red', 'green', 'blue']))	['r', 'g', 'b']
	map(func, i1, ..., ik)	Executes the function on all k elements of the k iterables	list(map(lambda x, y: str(x) + ' ' + y + 's', [0, 2, 2], ['apple', 'orange', 'banana']))	['0 apples', '2 oranges', '2 bananas']
	string.join(iter)	Concatenates iterable elements separated by string	'marries'.join(list(['Alice', 'Bob']))	'Alice marries Bob'
	filter(func, iterable)	Filters out elements in iterable for which function returns False (or 0)	list(filter(lambda x: True if x>17 else False, [1, 15, 17, 18]))	[18]
	string.strip()	Removes leading and trailing whitespaces of string	print("\n\t42\t".strip())	42
	sorted(iter)	Sorts iterable in ascending order	sorted([8, 3, 2, 42, 5])	[2, 3, 5, 8, 42]
	sorted(iter, key=key)	Sorts according to the key function in ascending order	sorted([8, 3, 2, 42, 5], key=lambda x: 0 if x==42 else x)	[42, 2, 3, 5, 8]
	help(func)	Returns documentation of func	help(str.upper())	'... to uppercase.'
	zip(i1, i2, ...)	Groups the i-th elements of iterators i1, i2, ... together	list(zip(['Alice', 'Anna'], ['Bob', 'Jon', 'Frank']))	[('Alice', 'Bob'), ('Anna', 'Jon')]
	Unzip	Equal to: 1) unpack the zipped list, 2) zip the result	list(zip(*(['Alice', 'Bob'], ('Anna', 'Jon'))))	[('Alice', 'Anna'), ('Bob', 'Jon')]
	enumerate(iter)	Assigns a counter value to each element of the iterable	list(enumerate(['Alice', 'Bob', 'Jon']))	[(0, 'Alice'), (1, 'Bob'), (2, 'Jon')]
TRICKS	python -m http.server <P>	Want to share files between PC and phone? Run this command in PC's shell. <P> is any port number 0–65535. Type <IP address of PC>:<P> in the phone's browser. You can now browse the files in the PC directory.		
	Read comic	import antigravity	Open the comic series xkcd in your web browser	
	Zen of Python	import this	'...Beautiful is better than ugly. Explicit is ...'	
	Swapping numbers	Swapping variables is a breeze in Python. No offense, Java!	a, b = 'Jane', 'Alice' a, b = b, a	a = 'Alice' b = 'Jane'
	Unpacking arguments	Use a sequence as function arguments via asterisk operator *. Use a dictionary (key, value) via double asterisk operator **	def f(x, y, z): return x + y * z f(*[1, 3, 4]) f(**{'z': 4, 'x': 1, 'y': 3})	13 13
	Extended Unpacking	Use unpacking for multiple assignment feature in Python	a, *b = [1, 2, 3, 4, 5]	a = 1 b = [2, 3, 4, 5]
	Merge two dictionaries	Use unpacking to merge two dictionaries into a single one	x={'Alice': 18} y={'Bob': 27, 'Ann': 22} z = {**x,**y}	z = {'Alice': 18, 'Bob': 27, 'Ann': 22}

Python Cheat Sheet: 14 Interview Questions

“A puzzle a day to learn, code, and play” → Visit finxter.com

Question	Code	Question	Code
Check if list contains integer x	<pre>l = [3, 3, 4, 5, 2, 111, 5] print(111 in l) # True</pre>	Get missing number in [1...100]	<pre>def get_missing_number(lst): return set(range(lst[len(lst)-1])[1:]) - set(l) l = list(range(1,100)) l.remove(50) print(get_missing_number(l)) # 50</pre>
Find duplicate number in integer list	<pre>def find_duplicates(elements): duplicates, seen = set(), set() for element in elements: if element in seen: duplicates.add(element) seen.add(element) return list(duplicates)</pre>	Compute the intersection of two lists	<pre>def intersect(lst1, lst2): res, lst2_copy = [], lst2[:] for el in lst1: if el in lst2_copy: res.append(el) lst2_copy.remove(el) return res</pre>
Check if two strings are anagrams	<pre>def is_anagram(s1, s2): return set(s1) == set(s2) print(is_anagram("elvis", "lives")) # True</pre>	Find max and min in unsorted list	<pre>l = [4, 3, 6, 3, 4, 888, 1, -11, 22, 3] print(max(l)) # 888 print(min(l)) # -11</pre>
Remove all duplicates from list	<pre>lst = list(range(10)) + list(range(10)) lst = list(set(lst)) print(lst) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]</pre>	Reverse string using recursion	<pre>def reverse(string): if len(string)<=1: return string return reverse(string[1:])+string[0] print(reverse("hello")) # olleh</pre>
Find pairs of integers in list so that their sum is equal to integer x	<pre>def find_pairs(l, x): pairs = [] for (i, el_1) in enumerate(l): for (j, el_2) in enumerate(l[i+1:]): if el_1 + el_2 == x: pairs.append((el_1, el_2)) return pairs</pre>	Compute the first n Fibonacci numbers	<pre>a, b = 0, 1 n = 10 for i in range(n): print(b) a, b = b, a+b # 1, 1, 2, 3, 5, 8, ...</pre>
Check if a string is a palindrome	<pre>def is_palindrome(phrase): return phrase == phrase[::-1] print(is_palindrome("anna")) # True</pre>	Sort list with Quicksort algorithm	<pre>def qsort(L): if L == []: return [] return qsort([x for x in L[1:] if x< L[0]]) + L[0:1] + qsort([x for x in L[1:] if x>=L[0]]) lst = [44, 33, 22, 5, 77, 55, 999] print(qsort(lst)) # [5, 22, 33, 44, 55, 77, 999]</pre>
Use list as stack, array, and queue	<pre># as a list ... l = [3, 4] l += [5, 6] # l = [3, 4, 5, 6] # ... as a stack ... l.append(10) # l = [4, 5, 6, 10] l.pop() # l = [4, 5, 6] # ... and as a queue l.insert(0, 5) # l = [5, 4, 5, 6] l.pop() # l = [5, 4, 5]</pre>	Find all permutations of string	<pre>def get_permutations(w): if len(w)<=1: return set(w) smaller = get_permutations(w[1:]) perms = set() for x in smaller: for pos in range(0,len(x)+1): perm = x[:pos] + w[0] + x[pos:] perms.add(perm) return perms print(get_permutations("nan")) # {'nna', 'ann', 'nan'}</pre>

Python Cheat Sheet: NumPy

“A puzzle a day to learn, code, and play” → Visit finxter.com

Name	Description	Example
<code>a.shape</code>	The shape attribute of NumPy array a keeps a tuple of integers. Each integer describes the number of elements of the axis.	<pre>a = np.array([[1,2],[1,1],[0,0]]) print(np.shape(a))</pre> <code># (3, 2)</code>
<code>a.ndim</code>	The ndim attribute is equal to the length of the shape tuple.	<pre>print(np.ndim(a))</pre> <code># 2</code>
<code>*</code>	The asterisk (star) operator performs the Hadamard product, i.e., multiplies two matrices with equal shape element-wise.	<pre>a = np.array([[2, 0], [0, 2]]) b = np.array([[1, 1], [1, 1]]) print(a*b)</pre> <code># [[2 0] [0 2]]</code>
<code>np.matmul(a,b)</code> , <code>a@b</code>	The standard matrix multiplication operator. Equivalent to the <code>@</code> operator.	<pre>print(np.matmul(a,b))</pre> <code># [[2 2] [2 2]]</code>
<code>np.arange([start,]stop, [step,])</code>	Creates a new 1D numpy array with evenly spaced values	<pre>print(np.arange(0,10,2))</pre> <code># [0 2 4 6 8]</code>
<code>np.linspace(start, stop, num=50)</code>	Creates a new 1D numpy array with evenly spread elements within the given interval	<pre>print(np.linspace(0,10,3))</pre> <code># [0. 5. 10.]</code>
<code>np.average(a)</code>	Averages over all the values in the numpy array	<pre>a = np.array([[2, 0], [0, 2]]) print(np.average(a))</pre> <code># 1.0</code>
<code><slice> = <val></code>	Replace the <code><slice></code> as selected by the slicing operator with the value <code><val></code> .	<pre>a = np.array([0, 1, 0, 0, 0]) a[::2] = 2 print(a)</pre> <code># [2 1 2 0 2]</code>
<code>np.var(a)</code>	Calculates the variance of a numpy array.	<pre>a = np.array([2, 6]) print(np.var(a))</pre> <code># 4.0</code>
<code>np.std(a)</code>	Calculates the standard deviation of a numpy array	<pre>print(np.std(a))</pre> <code># 2.0</code>
<code>np.diff(a)</code>	Calculates the difference between subsequent values in NumPy array a	<pre>fibs = np.array([0, 1, 1, 2, 3, 5]) print(np.diff(fibs, n=1))</pre> <code># [1 0 1 1 2]</code>
<code>np.cumsum(a)</code>	Calculates the cumulative sum of the elements in NumPy array a.	<pre>print(np.cumsum(np.arange(5)))</pre> <code># [0 1 3 6 10]</code>
<code>np.sort(a)</code>	Creates a new NumPy array with the values from a (ascending).	<pre>a = np.array([10,3,7,1,0]) print(np.sort(a))</pre> <code># [0 1 3 7 10]</code>
<code>np.argsort(a)</code>	Returns the indices of a NumPy array so that the indexed values would be sorted.	<pre>a = np.array([10,3,7,1,0]) print(np.argsort(a))</pre> <code># [4 3 1 2 0]</code>
<code>np.max(a)</code>	Returns the maximal value of NumPy array a.	<pre>a = np.array([10,3,7,1,0]) print(np.max(a))</pre> <code># 10</code>
<code>np.argmax(a)</code>	Returns the index of the element with maximal value in the NumPy array a.	<pre>a = np.array([10,3,7,1,0]) print(np.argmax(a))</pre> <code># 0</code>
<code>np.nonzero(a)</code>	Returns the indices of the nonzero elements in NumPy array a.	<pre>a = np.array([10,3,7,1,0]) print(np.nonzero(a))</pre> <code># [0 1 2 3]</code>



Real Python Pocket Reference

Visit realpython.com to turbocharge your Python learning with in-depth tutorials, real-world examples, and expert guidance.

Getting Started

Follow these guides to kickstart your Python journey:

- [realpython.com/what-can-i-do-with-python](#)
- [realpython.com/installing-python](#)
- [realpython.com/python-first-steps](#)

Start the Interactive Shell

```
$ python
```

Quit the Interactive Shell

```
>>> exit()
```

Run a Script

```
$ python my_script.py
```

Run a Script in Interactive Mode

```
$ python -i my_script.py
```

Learn More on [realpython.com/search](#):

[interpreter](#) · [run a script](#) · [command line](#)

Comments

- Always add a space after the #
- Use comments to explain “why” of your code

Write Comments

```
# This is a comment
# print("This code will not run.")
print("This will run.") # Comments are ignored by Python
```

Learn More on [realpython.com/search](#):

[comment](#) · [documentation](#)

Data Types

- Python is dynamically typed
- Use **None** to represent missing or optional values
- Use **type()** to check object type
- Check for a specific type with **isinstance()**
- issubclass()** checks if a class is a subclass

Type Investigation

```
type(42)           # <class 'int'>
type(3.14)         # <class 'float'>
type("Hello")     # <class 'str'>
type(True)        # <class 'bool'>
type(None)        # <class 'NoneType'>

isinstance(3.14, float) # True
issubclass(int, object) # True - everything inherits from object
```

Type Conversion

```
int("42")           # 42
float("3.14")       # 3.14
str(42)             # "42"
bool(1)             # True
list("abc")          # ["a", "b", "c"]
```

Learn More on [realpython.com/search](#):

[data types](#) · [type checking](#) · [isinstance](#) · [issubclass](#)

Variables & Assignment

- Variables are created when first assigned
- Use descriptive variable names
- Follow **snake_case** convention

Basic Assignment

```
name = "Leo"        # String
age = 7             # Integer
height = 5.6        # Float
is_cat = True       # Boolean
flaws = None        # None type
```

Parallel & Chained Assignments

```
x, y = 10, 20      # Assign multiple values
a = b = c = 0      # Give same value to multiple variables
```

Augmented Assignments

```
counter += 1
numbers += [4, 5]
permissions |= write
```

Learn More on [realpython.com/search](#):

[variables](#) · [assignment operator](#) · [walrus operator](#)

Strings

- It's recommended to use double-quotes for strings
- Use `"\n"` to create a line break in a string
- To write a backslash in a normal string, write `"\\"`

Creating Strings

```
single = 'Hello'
double = "World"
multi = """Multiple
line string"""
```

String Operations

```
greeting = "me" + "ow!" # "meow!"
repeat = "Meow!" * 3     # "Meow!Meow!Meow!"
length = len("Python")  # 6
```

String Methods

```
"a".upper()           # "A"
"A".lower()           # "a"
" a ".strip()         # "a"
"abc".replace("bc", "ha") # "aha"
"a b".split()         # ["a", "b"]
"-".join(["a", "b"])  # "a-b"
```

String Indexing & Slicing

```
text = "Python"
text[0] # "P" (first)
text[-1] # "n" (last)
text[1:4] # "yth" (slice)
text[:3] # "Pyt" (from start)
text[3:] # "hon" (to end)
text[::2] # "Pto" (every 2nd)
text[::-1] # "nohtyP" (reverse)
```

String Formatting

```
# f-strings
name = "Aubrey"
age = 2
f"Hello, {name}!" # "Hello, Aubrey!"
f"{name} is {age} years old" # "Aubrey is 2 years old"
f"Debug: {age=}" # "Debug: age=2"

# Format method
template = "Hello, {name}! You're {age}."
template.format(name="Aubrey", age=2) # "Hello, Aubrey! You're 2."
```

Raw Strings

```
# Normal string with an escaped tab
"This is:\tCool." # "This is: Cool."

# Raw string with escape sequences
r"This is:\tCool." # "This is:\tCool."
```

Learn More on [realpython.com/search](#):

[strings](#) · [string methods](#) · [slice notation](#) · [raw strings](#)

Numbers & Math

Arithmetic Operators

```
10 + 3 # 13
10 - 3 # 7
10 * 3 # 30
10 / 3 # 3.3333333333333335
10 // 3 # 3
10 % 3 # 1
2 ** 3 # 8
```

Useful Functions

```
abs(-5) # 5
round(3.7) # 4
round(3.14159, 2) # 3.14
min(3, 1, 2) # 1
max(3, 1, 2) # 3
sum([1, 2, 3]) # 6
```

Learn More on [realpython.com/search](#):

[math](#) · [operators](#) · [built in functions](#)

Conditionals

- Python uses indentation for code blocks
- Use 4 spaces per indentation level

If-Elif-Else

```
if age < 13:
    category = "child"
elif age < 20:
    category = "teenager"
else:
    category = "adult"
```

Comparison Operators

```
x == y # Equal to
x != y # Not equal to
x < y # Less than
x <= y # Less than or equal
x > y # Greater than
x >= y # Greater than or equal
```

Logical Operators

```
if age >= 18 and has_car:
    print("Roadtrip!")

if is_weekend or is_holiday:
    print("No work today.")

if not is_raining:
    print("You can go outside.")
```

Learn More on [realpython.com/search](#):

[conditional statements](#) · [operators](#) · [truthy falsy](#)

Loops

- `range(5)` generates 0 through 4
- Use `enumerate()` to get index and value
- `break` exits the loop, `continue` skips to next
- Be careful with `while` to not create an infinite loop

For Loops

```
# Loop through range
for i in range(5):    # 0, 1, 2, 3, 4
    print(i)
```

```
# Loop through collection
fruits = ["apple", "banana"]
for fruit in fruits:
    print(fruit)
```

```
# With enumerate for index
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

While Loops

```
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f"You entered: {user_input}")
```

Loop Control

```
for i in range(10):
    if i == 3:
        continue # Skip this iteration
    if i == 7:
        break     # Exit loop
    print(i)
```

Learn More on realpython.com/search:

[for loop](#) · [while loop](#) · [enumerate](#) · [control flow](#)

Functions

- Define functions with `def`
- Always use `()` to call a function
- Add `return` to send values back
- Create anonymous functions with the `lambda` keyword

Defining Functions

```
def greet():
    return "Hello!"

def greet_person(name):
    return f"Hello, {name}!"
```

```
def add(x, y=10):    # Default parameter
    return x + y
```

Calling Functions

```
greet()              # "Hello!"
greet_person("Bartosz") # "Hello, Bartosz"
add(5, 3)            # 8
add(7)               # 17
```

Return Values

```
def get_min_max(numbers):
    return min(numbers), max(numbers)

minimum, maximum = get_min_max([1, 5, 3])
```

Useful Built-in Functions

```
callable() # Checks if an object can be called as a function
dir()       # Lists attributes and methods
globals()  # Get a dictionary of the current global symbol table
hash()     # Get the hash value
id()       # Get the unique identifier
locals()   # Get a dictionary of the current local symbol table
repr()     # Get a string representation for debugging
```

Lambda Functions

```
square = lambda x: x**2
result = square(5) # 25

# With map and filter
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

Learn More on realpython.com/search:

[define functions](#) · [return multiple values](#) · [lambda](#)

Classes

- Classes are blueprints for objects
- You can create multiple instances of one class
- You commonly use classes to encapsulate data
- Inside a class, you provide methods for interacting with the data
- `__init__()` is the constructor method
- `self` refers to the instance

Defining Classes

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

# Create instance
my_dog = Dog("Frieda", 3)
print(my_dog.bark()) # Frieda says Woof!
```

Class Attributes & Methods

```
class Cat:
    species = "Felis catus" # Class attribute

    def __init__(self, name):
        self.name = name    # Instance attribute

    def meow(self):
        return f"{self.name} says Meow!"

    @classmethod
    def create_kitten(cls, name):
        return cls(f"Baby {name}")
```

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks!"
```

Learn More on realpython.com/search:

[object oriented programming](#) · [classes](#)

Exceptions

- When Python runs and encounters an error, it creates an exception
- Use specific exception types when possible
- `else` runs if no exception occurred
- `finally` always runs, even after errors

Try-Except

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print(f"Result: {result}")
finally:
    print("Calculation attempted")
```

Common Exceptions

```
ValueError      # Invalid value
TypeError       # Wrong type
IndexError      # List index out of range
KeyError        # Dict key not found
FileNotFoundError # File doesn't exist
```

Raising Exceptions

```
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age
```

Learn More on realpython.com/search:

[exceptions](#) · [errors](#) · [debugging](#)

Collections

- A collection is any container data structure that stores multiple items
- If an object is a collection, then you can loop through it
- Strings are collections, too
- Use `len()` to get the size of a collection
- You can check if an item is in a collection with the `in` keyword
- Some collections may look similar, but each data structure solves specific needs

Lists

```
# Creating lists
empty = []
nums = [5]
mixed = [1, "two", 3.0, True]
```

```
# List methods
nums.append("x")      # Add to end
nums.insert(0, "y")   # Insert at index 0
nums.extend(["z", 5]) # Extend with iterable
nums.remove("x")      # Remove first "x"
last = nums.pop()     # Pop returns last element
```

```
# List indexing and checks
fruits = ["banana", "apple", "orange"]
fruits[0]      # "banana"
fruits[-1]     # "orange"
"apple" in fruits # True
len(fruits)    # 3
```

Tuples

```
# Creating tuples
point = (3, 4)
single = (1,) # Note the comma!
empty = ()
```

```
# Basic tuple unpacking
point = (3, 4)
x, y = point
x      # 3
y      # 4
```

```
# Extended unpacking
first, *rest = (1, 2, 3, 4)
first        # 1
rest         # [2, 3, 4]
```

Sets

```
# Creating Sets
a = {1, 2, 3}
b = set([3, 4, 4, 5])

# Set Operations
a | b      # {1, 2, 3, 4, 5}
a & b      # {3}
a - b      # {1, 2}
a ^ b      # {1, 2, 4, 5}
```

Dictionaries

```
# Creating Dictionaries
empty = {}
pet = {"name": "Leo", "age": 42}

# Dictionary Operations
pet["sound"] = "Purr!" # Add key and value
pet["age"] = 7         # Update value
age = pet.get("age", 0) # Get with default
del pet["sound"]       # Delete key
pet.pop("age")         # Remove and return

# Dictionary Methods
pet = {"name": "Frieda", "sound": "Bark!"}
pet.keys() # dict_keys(['name', 'sound'])
pet.values() # dict_values(['Frieda', 'Bark!'])
pet.items() # dict_items([('name', 'Frieda'), ('sound', 'Bark!')])
```

Learn More on [realpython.com/search](#):
list · tuple · set · dictionary · indexing · unpacking

Comprehensions

- You can think of comprehensions as condensed for loops
- Comprehensions are faster than equivalent loops

```
# Basic
squares = [x**2 for x in range(10)]

# With condition
evens = [x for x in range(20) if x % 2 == 0]

# Nested
matrix = [[i*j for j in range(3)] for i in range(3)]
```

Other Comprehensions

```
# Dictionary comprehension
word_lengths = {word: len(word) for word in ["hello", "world"]}

# Set comprehension
unique_lengths = {len(word) for word in ["who", "what", "why"]}

# Generator expression
sum_squares = sum(x**2 for x in range(1000))
```

Learn More on [realpython.com/search](#):
comprehensions · data structures · generators

File I/O

File Operations

```
# Read an entire file
with open("file.txt", mode="r", encoding="utf-8") as file:
    content = file.read()

# Read a file line by line
with open("file.txt", mode="r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())

# Write a file
with open("output.txt", mode="w", encoding="utf-8") as file:
    file.write("Hello, World!\n")

# Append to a File
with open("log.txt", mode="a", encoding="utf-8") as file:
    file.write("New log entry\n")
```

Learn More on [realpython.com/search](#):
files · context manager · pathlib

- Imports & Modules
- Prefer explicit imports over `import *`
 - Use aliases for long module names
 - Group imports: standard library, third-party libraries, user-defined modules

Import Styles

```
# Import entire module
import math
result = math.sqrt(16)

# Import specific function
from math import sqrt
result = sqrt(16)

# Import with alias
import numpy as np
array = np.array([1, 2, 3])

# Import all (not recommended)
from math import *
```

Package Imports

```
# Import from package
import package.module
from package import module
from package.subpackage import module

# Import specific items
from package.module import function, Class
from package.module import name as alias
```

Learn More on [realpython.com/search](#):
import · modules · packages

Virtual Environments

- Virtual Environments are often called “venv”
- Use venvs to isolate project packages from the system-wide Python packages

Create Virtual Environment

```
$ python -m venv .venv
```

Activate Virtual Environment (Windows)

```
PS> .venv\Scripts\activate
```

Activate Virtual Environment (Linux & macOS)

```
$ source .venv/bin/activate
```

Deactivate Virtual Environment

```
(.venv) $ deactivate
```

Learn More on [realpython.com/search](#):
virtual environment · venv

- Packages
- The official third-party package repository is the Python Package Index (PyPI)

Install Packages

```
$ python -m pip install requests
```

Save Requirements & Install from File

```
$ python -m pip freeze > requirements.txt
$ python -m pip install -r requirements.txt
```

- Related Tutorials
- Installing Python Packages
 - Requirements Files in Python Projects

Miscellaneous	
Truthy	Falsy
-42	0
3.14	0.0
"John"	""
[1, 2, 3]	[]
("apple", "banana")	()
{"key": None}	{}
	None

Pythonic Constructions

```
# Swap variables
a, b = b, a

# Flatten a list of lists
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [item for sublist in matrix for item in sublist]

# Remove duplicates
unique_unordered = list(set(my_list))

# Remove duplicates, preserve order
unique = list(dict.fromkeys(my_list))

# Count occurrences
from collections import Counter

counts = Counter(my_list)
```

Learn More on [realpython.com/search](#):
counter · tricks

Do you want to go deeper on any topic in the Python curriculum?

At Real Python you can immerse yourself in any topic. Level up your skills effectively with curated resources like:

- Learning paths
- Video courses
- Written tutorials
- Interactive quizzes
- Podcast interviews
- Reference articles

Continue your learning journey and become a Python expert at [realpython.com/start-here](#)  